

Entorno de Desarrollo y Ejemplos

Presentador: Ariel Gámez



Desarrollo Blockchain

Docker + Remix Desktop

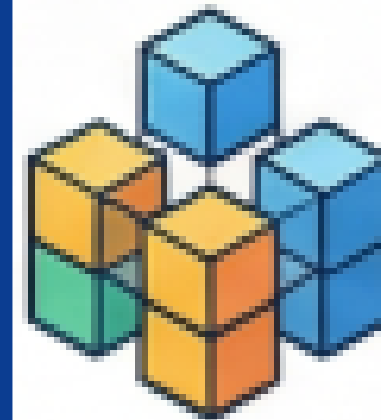
Enfoque Visual e Interactivo



Ideal para aprender, prototipar rápidamente y depurar visualmente una función específica.



Interacción
GUI (clics y botones)



Reproducibilidad
Baja (procesos manuales)



+



+



Caso de Uso Ideal
Aprender y depurar visualmente

Docker + Foundry Scripts

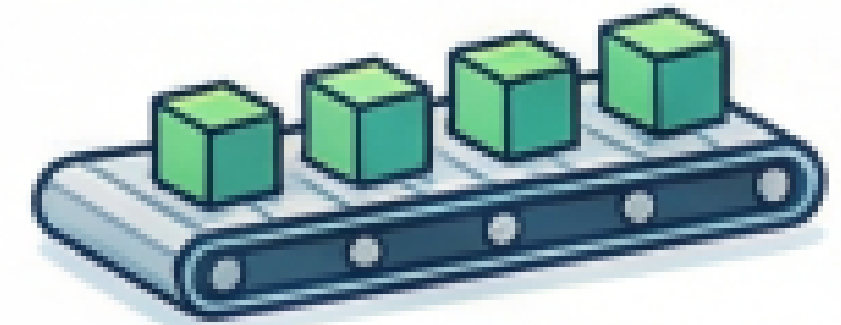
Enfoque de Ingeniería “Code-First”



Estándar para desarrollos profesionales, testing complejo y despliegues automatizados y reproducibles.



Interacción
CLI (comandos y scripts)



Reproducibilidad
Alta (todo es código)



Caso de Uso Ideal
Tests, CI/CD y sistemas complejos

Desarrollo Local

Preferencia Anvil (vs. Geth)



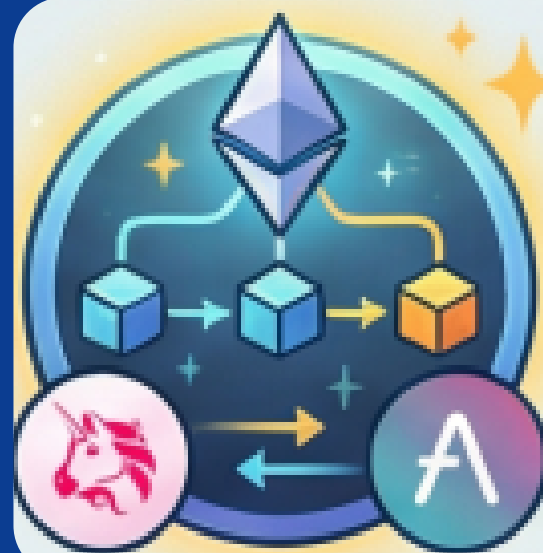
Minado Instantáneo para Tests Ultrarrápidos

Anvil crea bloques bajo demanda en milisegundos, a diferencia de Geth que simula tiempos de espera reales.



"Modo Dios": Control Total sobre la Blockchain

Permite "viajar en el tiempo" (`vm.warp`) y suplantar cuentas (`vm.prank`) para pruebas avanzadas.



Forking de Mainnet: La "Killer Feature"

Clona el estado real de Ethereum para interactuar con contratos como Uniswap o Aave en tu entorno local.

Ejemplos Prácticos

De Scripts Simples a Arquitecturas IoT

```

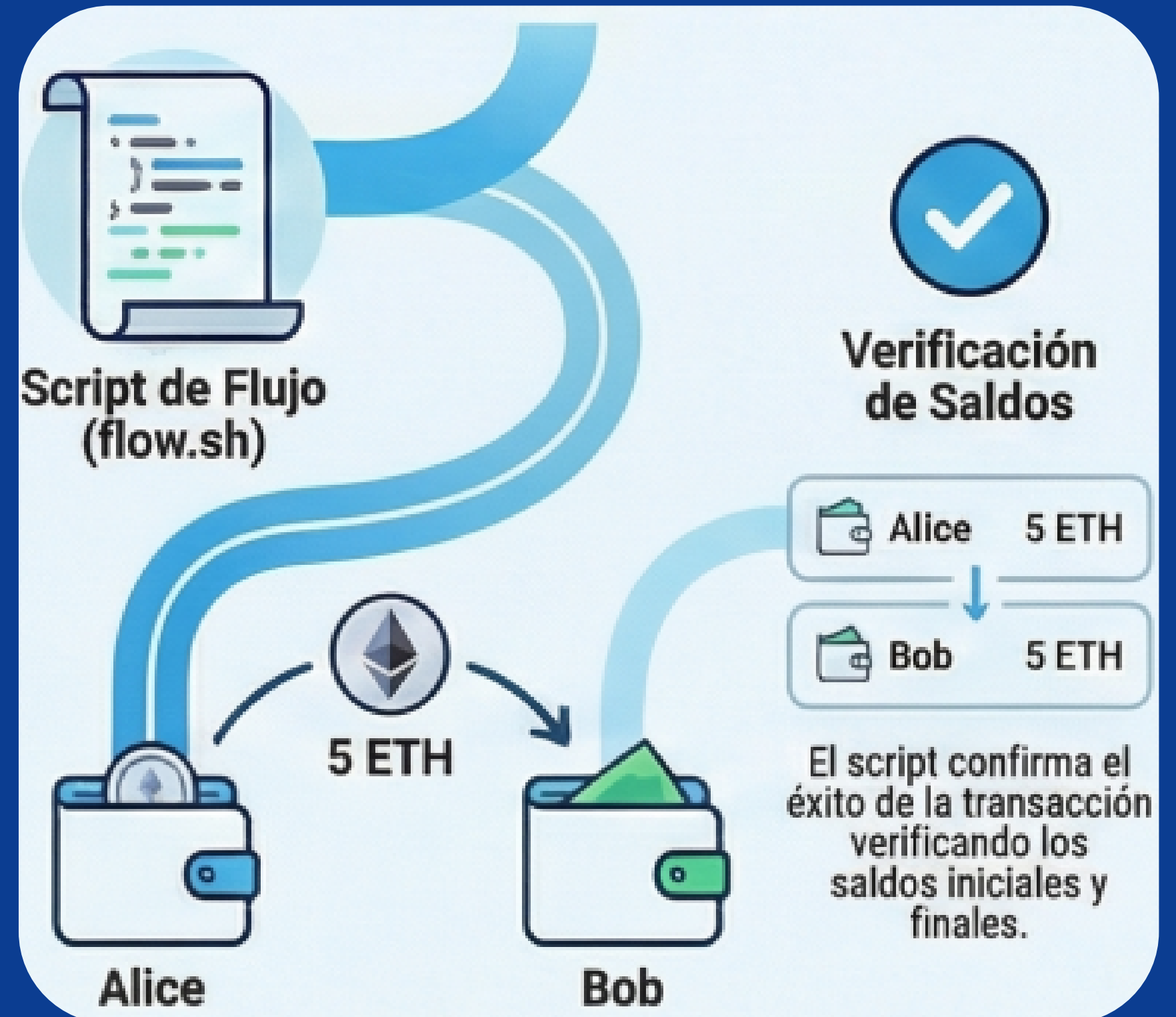
11 public con/ ftrwvt eroticization: mendats ateat {/}
12 nl -/metin(fibe: copthnom atmczl-indent 4});
10 ar -{thmmulig → {public {}
17 fill → elitz → rXpades Agem()
13 nl -(nodal({} → l3m → p̄roternurach.
14 public inget e...entlate ,>
16 arorel...
17
11 from it bitd: {}
18 nl -cervatxae colvelemen,
19 p/l -afuly xazeen <r
{

15 -public every toes chock {bit publics
17 ar -dst{←≡, code,}
18 fill =_public,
14 px -fart TX))
13
12 padtyllte: toire, = privet
13 {
16 fill -/nereditlor eub))
{
-publics
eat0(t,sif)
)
}

```

NIVEL 1

Automatización Básica



Automatización de una Transferencia Simple

`Dockerfile`

```
FROM ghcr.io/foundry-rs/foundry:stable

WORKDIR /app
COPY flow.sh .
RUN chmod +x flow.sh
```

`flow.sh`

```
#!/bin/sh
BOB_ADDRESS=$(docker exec red-local cast wallet
  new | awk '/Address:/ {print $2}')
echo "Verifying initial balance..."
docker exec red-local cast balance $BOB_ADDRESS
echo "Sending 5 ETH..."
docker exec red-local cast send \
  --private-key $ALICE_PK $BOB_ADDRESS 5ether
echo "Verifying final balance..."
docker exec red-local cast balance $BOB_ADDRESS
```

NIVEL 2

Proyecto Completo y Reproducible



Un Contrato, Un Script, Cero Sorpresas

Flujo de Configuración

1. → forge init
↓
2. → forge-std install
↓
3. → foundry.toml config
↓
4. → forge build
↓
5. → forge script Interact.s.sol

```
script/Interact.s.sol

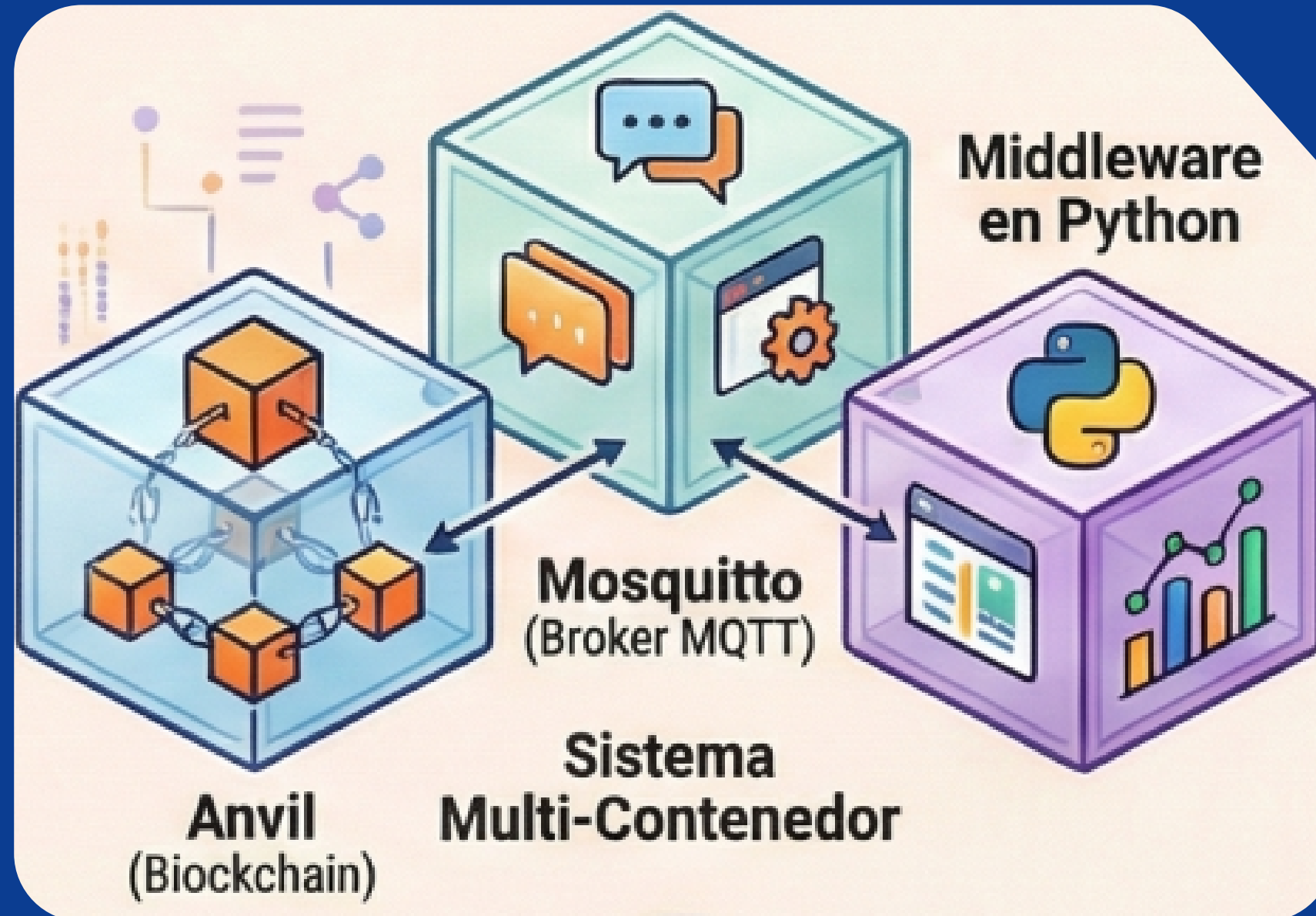
// ...
PiggyBank piggyBank = new PiggyBank();

// Owner deposits
vm.startBroadcast(owner); ←
piggyBank.deposit{value: 1 ether}();
vm.stopBroadcast();

// User tries to withdraw
vm.startBroadcast(user); ←
piggyBank.withdraw();
vm.stopBroadcast();
// ...
```

NIVEL 3

Arquitectura IoT + Blockchain



Flujo de Datos en Acción

```
> Publicando JSON en sensor/data...  
> {"sensor": "T-01", "temp": 25.4, "ts": 1677612345}  
> ... esperando confirmación ...  
>  Confirmación Blockchain recibida! Tx Hash: 0x1a2b3c...
```

1. **ESP32 publica datos**
→ MQTT
2. **Python firma y envía TX**
→ Anvil
3. **Smart Contract guarda datos inmutables**
4. **Hash de confirmación vuelve al ESP32**

Conclusión y Recursos